Высшая
школа
бизнеса

Business Analytics and Big Data Systems

Moscow
May 2026

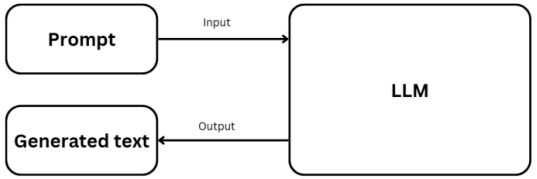
Automatically Evolving Multi-Turn Customer Service AI Agents via Prompt-Level Distillation

Author: Gleb Dementev
Supervisor: Seungmin Jin, PhD

Key Concepts

Large Language Model

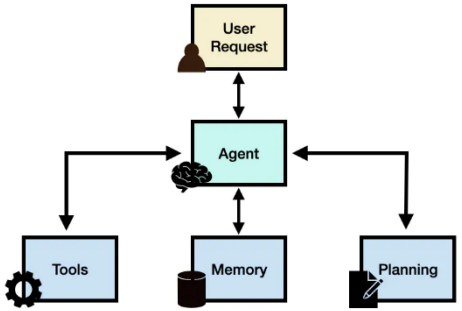
Neural network that generates plausible text from learned distributions



GPT-5.4, Opus 4.6, Gemini 3.1

Tool-Calling Agent

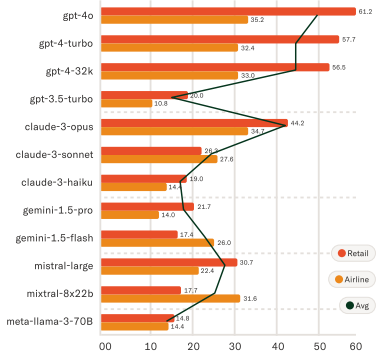
LLM harness that allows calls to external tools via structured text output



Claude Code, OpenAI Codex, Manus

Agentic Benchmark

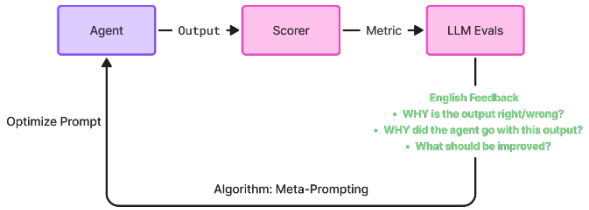
LLM performance measurement on tasks requiring planning and action



Tau-2, Toolbench, GAIA

Prompt Evolution

Automated benchmark pass rate improvement



GEPA, DSPy, EvoPrompt

Relevance & Research Gap

1. Low Agentic Benchmark Pass Rates

Enterprises cannot do with 98.5% pass rate -> instant 1.5pp churn increase

2. Other benchmarks show promising techniques

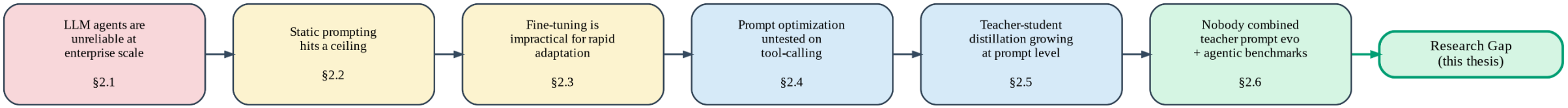
Prompt Evolution and teacher-student distillation have been shown to improve performance across domains

Question: Can we automatically improve agentic performance at prompt level?

Figure 2.5: Prompt Optimization Methods and Evaluation Domain Coverage

	QA / Reasoning	Math	Coding	Classifi-cation	Interactive Env	Tool-Calling Benchmark
DSPy	Y	Y				–
TextGrad	Y		Y			–
OPRO	Y	Y				–
EvoPrompt	Y			Y		–
PromptBreeder	Y			Y		–
ProTeGi				Y		–
Reflexion	Y		Y		Y	–
ADAS	Y				Y	–
Trace	Y					–
AgentOptimizer					Y	–
AvaTaR	Y					–
GEPA	Y	Y				–
SCOPE					Y	–
LEAP					Y	–

No existing work validated here



Problem, Goal, Questions, Tasks

Problem: LLM agents are unreliable at enterprise scale and tasks, as shown by agentic benchmarks

Goal: Demonstrate automatic benchmark pass rate improvement and show its limitations

Q1

Which failure modes respond to prompt/schema edits? Which resist?

Q2

How well does fix rate respond to task variety?

Q3

Can we close the gap to frontier model w/o weight updates?

Objective	Outcome
1. Baseline eval on τ^2-bench airline	Failure mode catalogue
2. Diagnose-patch-validate loop	Evolution framework (software)
3. Eval across 5/10/20 tasks, 3 models	Scaling data + cross-model comparison
4. Measure gap closure	Primary outcome metric
5. Analyse patch distribution	Boundaries of prompt-level intervention

Literature Review: Why Prompt-Level Evolution?

1. Agents fail

LLM agents are far below enterprise reliability thresholds

tau-bench (Yao et al., 2024)
GPT-4o fails >50% of tasks

GAIA (Mialon et al., 2023)
humans 92% vs GPT-4 15%

SWE-bench (Jimenez et al., 2024)
best model solves <2%

ToolBench (Qin et al., 2023)
OSS models score 0%

Agents need to get better.
Can we just prompt-engineer?

2. Prompting is brittle

Static prompts help but are fragile and hit a hard ceiling

Sensitivity (Sclar et al., 2023)
format → 76 pts variation

CoT (Wei et al., 2022)
only works at ≥100B

ReAct (Yao et al., 2023)
gains, but no self-repair

APE (Zhou et al., 2022)
optimal prompts are fragile

Manual prompts can't adapt.
Can we fine-tune instead?

3. Fine-tuning is costly

Weight modification works but is expensive and destructive

Alignment tax (Young, 2026)
proven irreducible

SFT harm (Shenfeld, 2026)
collapses reasoning

LIMA (Zhou et al., 2023)
alignment = style, not knowledge

InstructGPT (Ouyang et al., 2022)
1.3B>175B, but regresses

Weights are the wrong level.
Can we optimize prompts auto-matically?

4. Auto-optimization exists

Prompt optimizers show strong gains — but not on agents

DSPy (Khattab et al., 2023)
+5–46% on HotPotQA

TextGrad (Yuksekgonul, 2024)
+20% LeetCode; Nature

EvoPrompt (Guo et al., 2023)
+25% Big-Bench Hard

OPRO (Yang et al., 2023)
+50% Big-Bench Hard

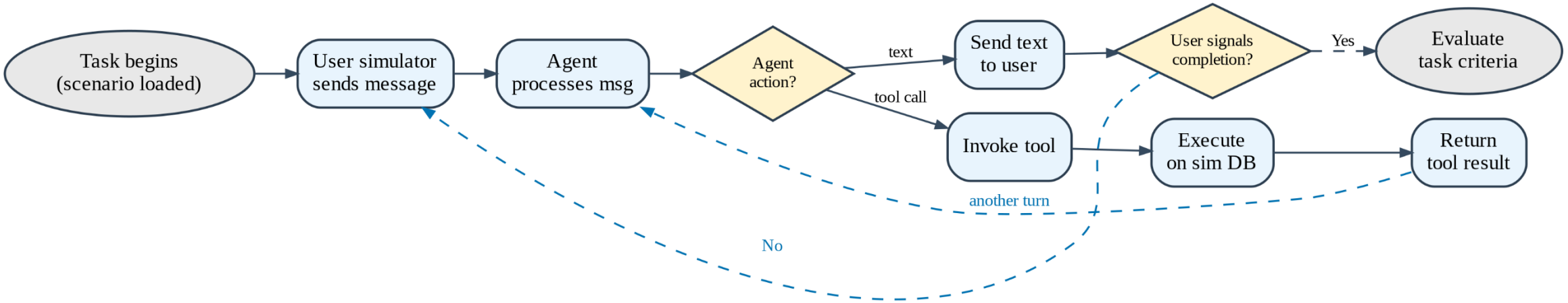
Tools exist, benchmarks exist, but the two have not met.

Dataset: tau2-bench

τ^2-bench (2025): customer service automation agents with pre-existing prompts, no saturation, tool-calls, complex failure modes

We picked airline domain, it had 50 tasks (subsets: 5, 10, 20)

Tau-2 benchmark architecture

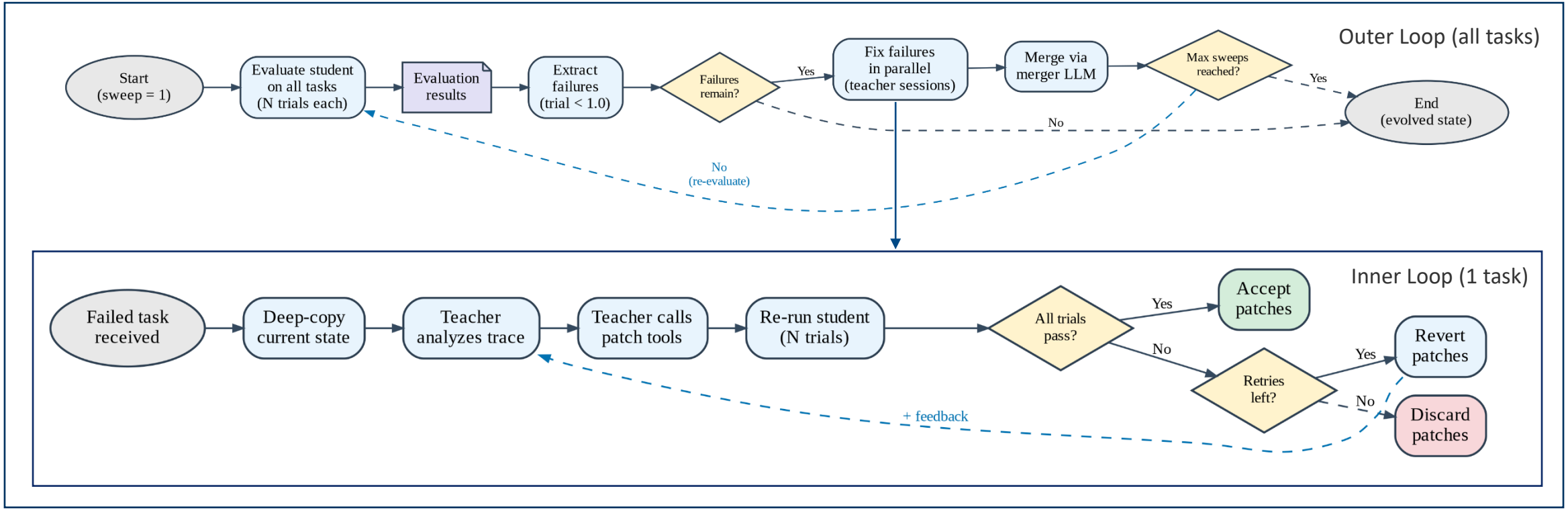


We chose SOTA OSS for benchmarking. Student models had to be non-thinking.

Role	Model	Rationale
Student	Qwen3 30B-A3B	Cost-efficient MoE; headroom to frontier
Student	Qwen3.5 Flash	SOTA instant-response Qwen
Student	GLM 4.7 Flash	Different arch family, comparable benchmarks results
Teacher	Kimi K2.5 (~1T MoE)	Frontier reasoning

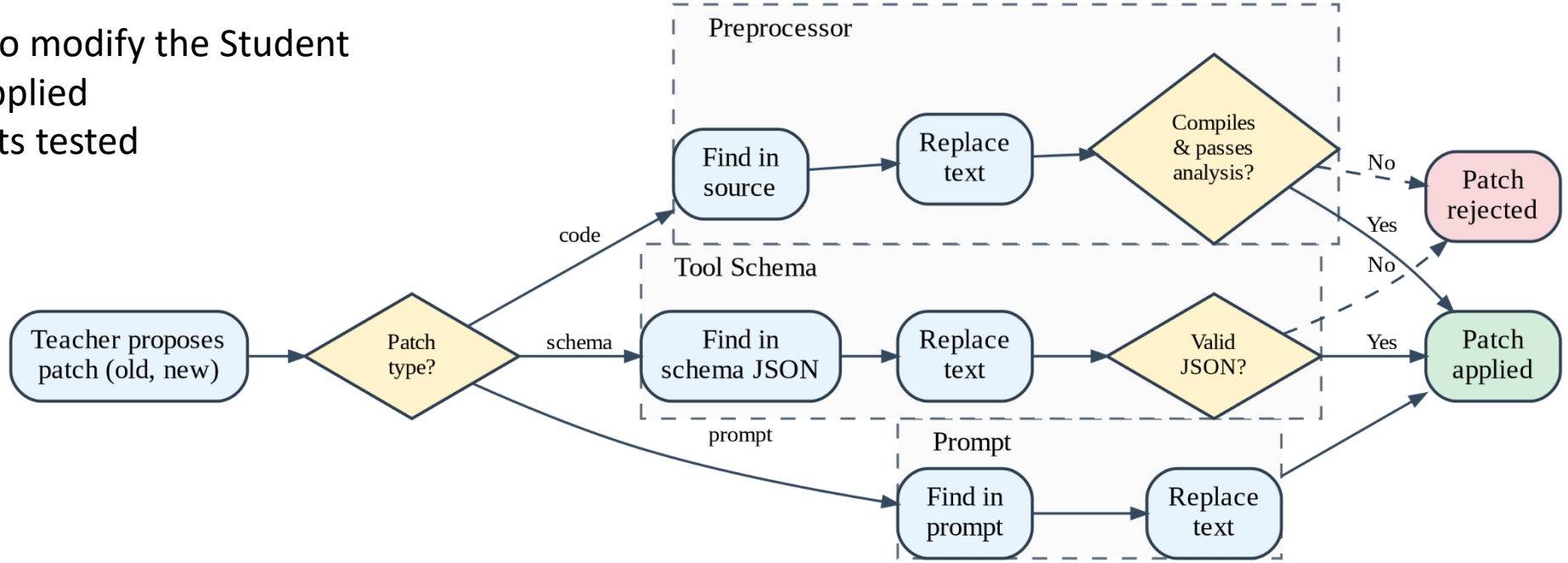
We had a teacher look at each error, fix it, then merge its fixes into global prompt

1. Run benchmark → 2. Find errors → 3. Try fixing each error → 4. Merge fixes → 5. Re-run benchmark



A teacher could edit instruction, tool schema and build validators

1. Benchmark evaluates task
2. Benchmark classifies a mistake
3. Benchmark gives Teacher the failure and its explanation
4. Teacher calls tools to modify the Student
5. Valid patches get applied
6. Patched Student gets tested



We built an experimental UI

20-task run summary

SWEEP RESULTS Per-task fix attempts across sweeps

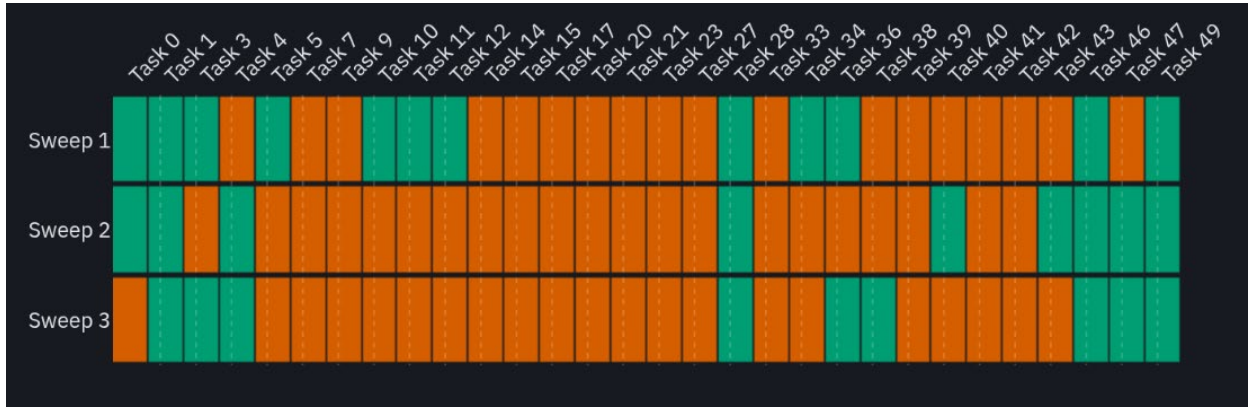
#	TASK	BASE	PATCH	DELTA	STATUS	TIER	TRY	MSGS	CALLS
1	7	Fail	Fail	0→0	NOT FIXED	—	—	26	10
1	11	Fail	Fail	0→0	NOT FIXED	—	—	56	24
1	10	Fail	Pass	0→1	FIXED	instruction	2	18	7
1	5	Fail	Fail	0→0	NOT FIXED	—	—	44	18
1	1	Fail	Pass	0→1	FIXED	guardrail	3	18	6
1	12	Fail	Fail	0→0	NOT FIXED	—	—	56	24
1	9	Fail	Fail	0→0	NOT FIXED	—	—	54	21
2	4	Fail	Pass	0→1	FIXED	instruction	1	8	3
2	1	Fail	Pass	0→1	FIXED	instruction	1	13	5
2	7	Fail	Fail	0→0	NOT FIXED	—	—	45	20
2	12	Fail	Fail	0→0	NOT FIXED	—	—	71	30
2	11	Fail	Fail	0→0	NOT FIXED	—	—	43	19
2	10	Fail	Fail	0→0	NOT FIXED	—	—	45	20

LLM teacher generating a fix

```
Let me fix this by adding clear instructions to the system prompt about the proper workflow for modifying flights.

TOOL CALL patch_prompt functions.patch_prompt:0
{
  "old_text": "## Modify flight\n\nFirst, the agent must obtain the user id and reservation id. \n- The user must provide their user id. \n- If the user doesn't know their reservation id, the agent should help locate it using available tools.",
  "new_text": "## Modify flight\n\nFirst, the agent must obtain the user id and reservation id. \n- The user must provide their user id. \n- If the user doesn't know their reservation id, the agent should help locate it using available tools.\n\n**Workflow for modifying flights:**\n1. Retrieve the user's details to get
```

20-task over 3 sweeps result heatmap



We have shown pass rate improvement on most student models

1. Qwen 30 shows most consistent improvement from weak baseline
2. Qwen 3.5 shows high baseline, less prompt-driven improvement
3. GLM-4.7 regresses after applying patches (tested 3 times)

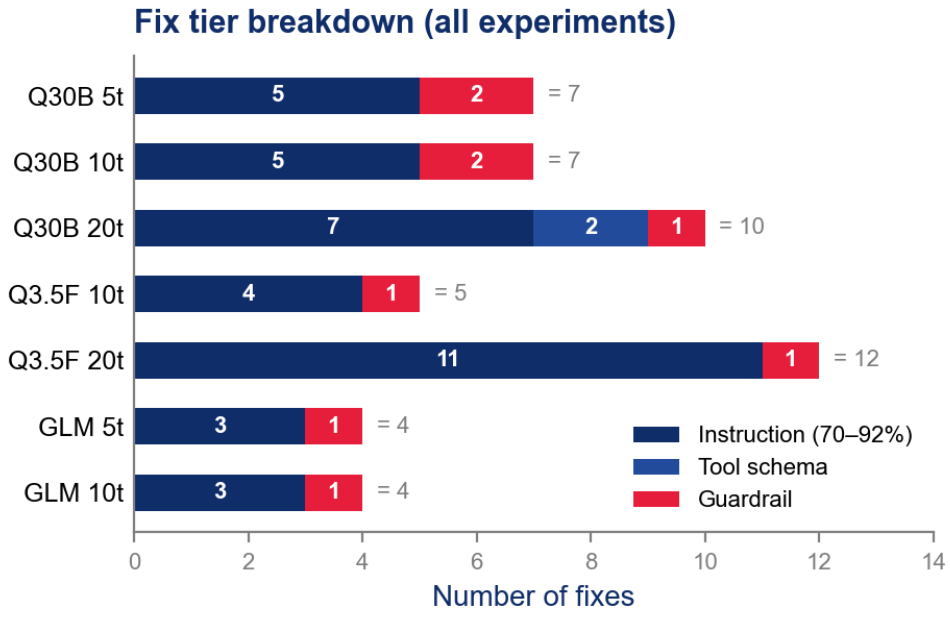
Task count	Qwen 30	Qwen 3.5	GLM-4.7
5	53 -> 73	100 -> N/A	47 -> 47
10	27 -> 50	60 -> 63	50 -> 40
20	22 -> 30	47 -> 58	N/A -> N/A

Q1: Which failure modes respond to prompt/schema edits? Which resist?

F1: We can fix only some tasks; others prove resistant

	Qwen3 30B-A3B		Qwen3.5 Flash		GLM 4.7 Flash	
Task	Base	Evolved	Base	Evolved	Base	Evolved
0	1/3	3/3	3/3	3/3	3/3	2/3
1	2/3	2/3	3/3	3/3	2/3	2/3
3	0/3	0/3	3/3	3/3	3/3	2/3
4	3/3	2/3	3/3	3/3	3/3	2/3
5	2/3	3/3	2/3	3/3	2/3	1/3
7	0/3	0/3	0/3	0/3	0/3	0/3
9	0/3	0/3	0/3	0/3	0/3	0/3
10	2/3	1/3	3/3	3/3	2/3	2/3
11	1/3	0/3	1/3	2/3	0/3	1/3
12	0/3	0/3	0/3	3/3	0/3	0/3
14	0/3	0/3	0/3	0/3	N/A	N/A
15	0/3	1/3	1/3	1/3	N/A	N/A
17	0/3	0/3	2/3	3/3	N/A	N/A
20	0/3	0/3	2/3	2/3	N/A	N/A
21	0/3	0/3	1/3	2/3	N/A	N/A
23	0/3	0/3	0/3	0/3	N/A	N/A
27	0/3	0/3	0/3	3/3	N/A	N/A
28	0/3	3/3	3/3	3/3	N/A	N/A
33	0/3	1/3	0/3	0/3	N/A	N/A
34	2/3	2/3	1/3	1/3	N/A	N/A

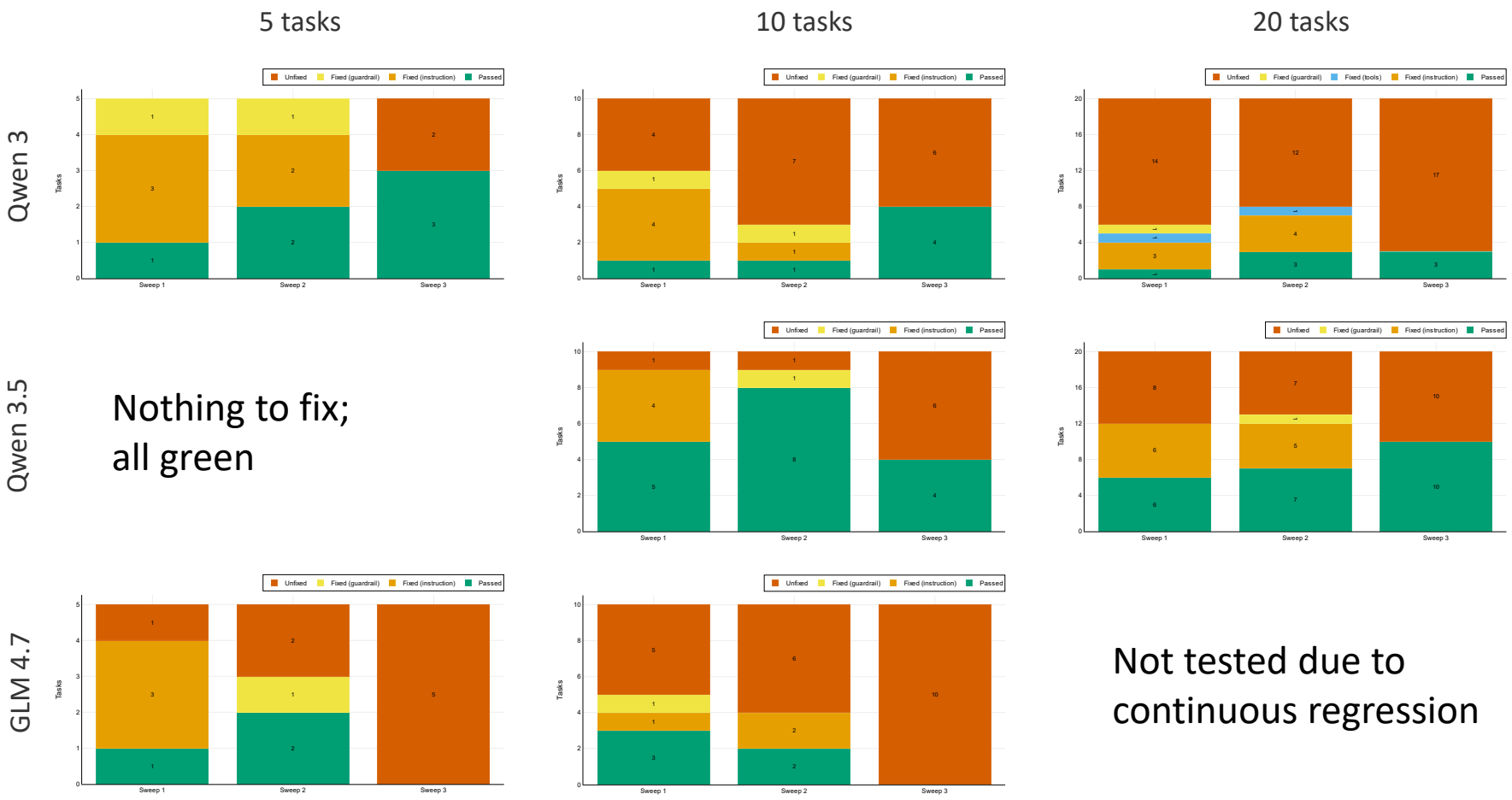
1. Many tasks are fully resistant to prompt modification. This implies that we hit model reasoning capability ceiling
2. Most fixes are powered by instruction (reward hacking is prevented). Guardrail fixes are mostly applied at later stages



Q2: How well does fix rate respond to task variety?

F2: More tasks -> lower the fix rate, but with caveats

1. Some models just can't get any benefits from prompt tuning – e.g. GLM 4.7
2. All models poorly retain of pass rate improvement - sweep 1 has 100% fix rate, only half persist to sweep 3
3. The stronger the model, the more it retains -> Qwen 3.5 is superior to Qwen 3



Q3: Can we close the gap to frontier model w/o weight updates?

F3: We cannot fully close the gap to frontier model

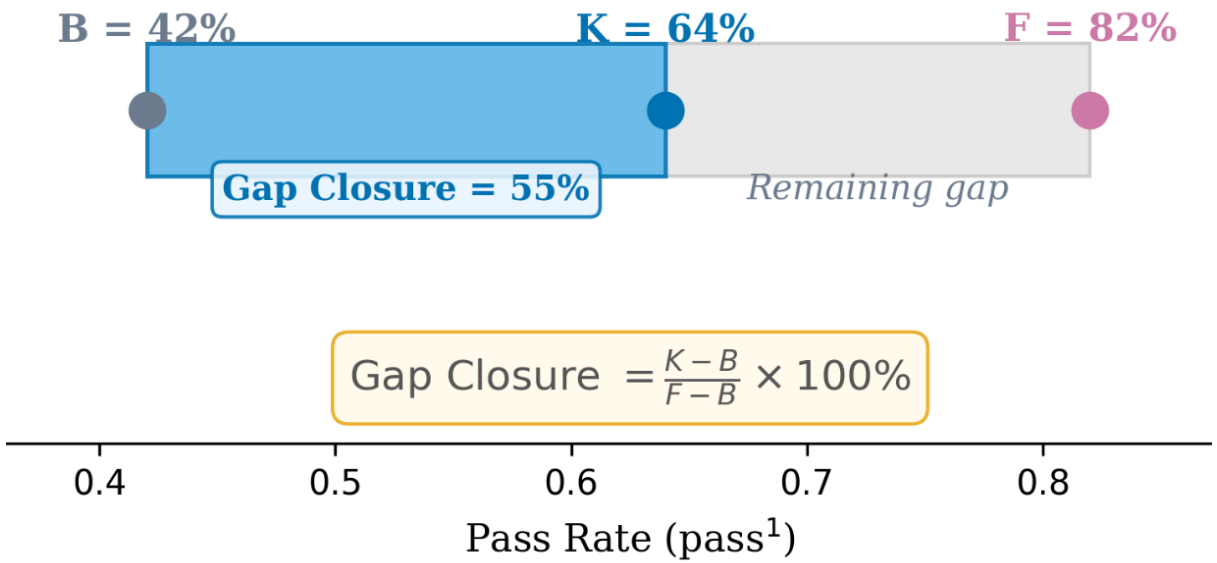
1. No model demonstrated full gap closure. Further modifications to the teaching pipeline required
2. Cannot yet infer any pattern what drives more/less gap closure

Baseline – Student, no prompt tuning

K (evolved) – Student after prompt tuning

Frontier – Teacher, no prompt tuning

Model	Scale	B (%)	K (%)	Δ (pp)	GC (%)
Qwen3 30B-A3B	5t	53	73	+20	**69%**
Qwen3 30B-A3B	10t	27	50	+23	**42%**
Qwen3 30B-A3B	20t	22	30	+8	**13%**
Qwen3.5 Flash	5t	100	100	0	N/A
Qwen3.5 Flash	10t	60	63	+3	**14%**
Qwen3.5 Flash	20t	47	58	+11	**31%**
GLM 4.7 Flash	5t	47	47	0	**0%**
GLM 4.7 Flash	10t	50	40	-10	**-31%**



Outcomes

Question	Result	
Q1: Which failure modes respond?	Policy failures most responsive. Multi-step reasoning resists intervention.	✓
Q2: Does fix rate scale?	Stable at 5–10 tasks (+20pp). Halves at 20 (+11pp). Capacity ceiling: 5–8 fixable tasks.	✓
Q3: Can patches close gap?	Yes for capable students (Qwen3.5→65%). No for weak (GLM→0%). Contingent on student.	~

Contributions

1. Teacher-driven prompt evolution framework — 3 patch surfaces, validate-before-merge

2. Empirical eval of auto prompt optimization on multi-turn tool-calling benchmark

3. Scaling evidence: stable gain, declining fix rate, hard core, student capability role

Limitations

- Single domain tested; only 20 tasks x 3 trials
- Only OSS models
- No RLHF, LoRA comparisons
- Insufficient reliability: 73% pass rate << 99.99%

Further Work

- Vary teachers and students
- Compare against adapted GEPA & others
- Multi-agent decomposition
- Test against other benchmarks